

P3696R0

Discovering Header Units Via Module Maps

Published Proposal, 2025-05-15

This version:

<http://wg21.link/P3696R0>

Author:

[Michael Spencer](#) (Apple)

Audience:

SG15

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

It's important that tools agree about which headers are header units, but we currently have no portable and reliable way to tell tools about this. This paper proposes adopting module maps, files co-located with headers, to resolve this problem.

Table of Contents

1 The Problem

2 Header Units and Clang Modules

2.1 Module Maps

2.2 What Makes a Header a Header Unit?

3 The Proposal

3.1 What Format?

- 3.2 /usr/include
- 3.3 3rd Party Headers
- 3.4 Build Systems
- 3.5 Special Cases
- 3.6 Further Extensions

4 Impact on The Standard

References

Informative References

§ 1. The Problem

C++20 introduced header units, but we still don't have great answers to which headers should be treated as header units, and when should include translation occur. It's important for all tools to agree on this to ensure they have the same view of the code and to avoid ODR issues. Today this is largely build system specific, and it's unclear who is responsible for making the decision.

§ 2. Header Units and Clang Modules

Header units were added to C++20 modules as part of the Merging Modules proposal [\[P1103R3\]](#) which combined Another Take On Modules [\[P0947R1\]](#) with the Modules TS (ISO/IEC TS 21544:2018). They were based on Clang's experience with [Clang header modules](#) including how Clang does include translation, but only minimal support in the standard was added. Clang header modules are an implementation of header units.

Over the past decade, the LLVM community has gained a lot of experience with Clang modules and how to build them. Today Apple, CERN, Google, Meta, and other organizations use Clang modules in production on hundreds of millions of lines of code, and have learned a lot along the way. One thing in particular that I believe would be useful for the C++ standard is module maps.

§ 2.1. Module Maps

In Clang modules, a module is defined by a module map. This allows grouping a set of headers together and building them equivalent to the way a header unit is built.

```
module M {           // All modules must have a name instead of all being part of
                    // global module.
    header "M.h"
    export *          // Other modules this module imports are not re-exported by
                    // default, you have to specify that.
}
```

An important part about module maps is how they are discovered. You can either pass them directly to the compiler on the command line, or they are located next to the headers they reference implicitly. With implicit module map search, tools can very quickly discover if a header belongs to a Clang module, and they all can agree on it. There are a lot of other features of Clang modules, but they aren't needed for the current scope of C++ header units.

§ 2.2. What Makes a Header a Header Unit?

A header is a good candidate to be a header unit if:

- It can be included in isolation into a TU
- The defines and declarations visible after its inclusion are not intended to depend on any previous preprocessor state
 - It can be included multiple times without changing meaning
 - No `#define` followed by `#include` where the `#define` can be different in different TUs
- It "owns" all of its defining declarations

This excludes several common headers in C such as `<assert.h>` and `<stddef.h>` (due to `__need_*` macros).

These properties can be somewhat subjective, and from our experience it's not always easy for developers to tell. This is particularly worse when the header in question isn't even your header, and when `#include_next` is involved.

The best person to determine if and when a header should be treated as a header unit is the person that wrote the header.

§ 3. The Proposal

C++ should adopt the concept of module maps as a common way to declare which headers are eligible to be header units, libraries should ship these files along side their headers, and tools should look for them.

§ 3.1. What Format?

Clang’s current module map format has worked well, but there are a few properties that make it difficult to just adopt into C++. It currently requires a name for each header unit, and doesn’t re-export declarations by default. Due to this, I’m not proposing any specific syntax at this time. Clang will be able to adopt whatever format C++ uses as long as it’s not ambiguous with its own.

It’s preferred to use the same `module.modulemap` filename, because searching for several alternative names can impose noticeable overhead in some cases, though this is not required. If a different name is chosen then the ambiguity concerns also go away.

§ 3.2. `/usr/include`

Nobody owns `/usr/include`, and so having a single module map file there will not work. As a solution to this, and a few other issues we’ve encountered, module maps will support having the module map file being a directory instead. In this case all of the module map files in that directory will be combined.

§ 3.3. 3rd Party Headers

Sometimes you have headers that you don’t own, but still need to treat as header units. For these cases Clang supports two different options. A module map can specify a header by absolute path and you can directly pass in the module map. Another solution, and the one I prefer, is to use Clang’s virtual file system to inject the module map into the needed path. This allows Clang’s normal module map discovery to find the module map, and it’s the same content as should eventually be provided by the header author.

§ 3.4. Build Systems

Not all build systems support discovering new work during the build, and this has been somewhat of a blocker for adopting header units into these build systems. With module maps the build system can discover the possible headers units at configure time, and then use them at build time the same way they would a named module. It would be very easy to pass these in directly, or discover them recursively from the include search path.

§ 3.5. Special Cases

There are named modules which are implemented by including headers into the body of a named module with a special macro defined to change their mode to allow this to work. Generally this would make the header not a candidate to be a header unit according to our rules above, but this is done for some standard library implementations where those same headers are required by the standard to be importable as header units. To support this use case, we can extend the module map format with a feature to say that a header should be treated as a header unit everywhere except when building some specific named module.

This allows supporting these use cases with no special build system work as is required today.

§ 3.6. Further Extensions

There are two extensions that I think would be quite useful immediately. The first is support for combining multiple headers into a single header unit. When deploying Clang modules people quickly learn that having one module per header does not scale. You quickly end up with 10s to 100s of thousands of modules, and no part of the system likes this. Today Clang supports combining headers and considers this to be a conforming extension due to header search being implementation defined. Clang treats a search for `<header.h>` to instead find `<__umbrella.h>` that actually includes `<header.h>` along with a few other headers. I believe other implementations have discovered this too and have already implemented similar extensions.

The second is redirecting an import. Under the same justification as the above, a search for the `<vector>` header could instead find a header that contains:

```
import std;
#include <macros_from_std>
```

This can ease the transition to named modules in some cases, and standardizing module maps would make this type of thing portable. I believe this has the same effect as in [P3041R0] - Transitioning from "#include" World to Modules.

§ 4. Impact on The Standard

This proposal would add a specific module map format and (approximate) lookup rules to the standard. I believe this belongs in the standard as it needs to be portable and the same across all implementations, and is conceptually part of the header.

§ References

§ Informative References

[P0947R1]

Richard Smith. *Another take on Modules*. 6 March 2018. URL: <https://wg21.link/p0947r1>

[P1103R3]

Richard Smith. *Merging Modules*. 22 February 2019. URL: <https://wg21.link/p1103r3>

[P3041R0]

Gabriel Dos Reis. *Transitioning from "#include" World to Modules*. 16 November 2023. URL: <https://wg21.link/p3041r0>